Bicol University POLANGUI CAMPUS

Centro Occidental Polangui, Albay

IT112: WEB SYSTEMS and TECHNOLOGIE'S

2nd Semester, AY: 2025-2026

Phase 4: Final Group Project

BentaBoard

"From bawat benta to tunay na kita—track it right."

Proposed By:

PeNiHAMiNo

BSIT 2A

Mantes, Michael B.

Obidece, Hazel Ann R.

Odo, Pearl Anne R.

Racadag, Nica Alyssa R.

Remolar, Noe Jr H.

Presented to:

Dr. Guillermo V. Red, Jr.

IT112 Professor

Introduction

In Phase 4 of our project, our group, **PeNiHAMiNo,** focused on the most critical part of the development process: connecting our frontend (the part the user sees) to our backend (the server and database). In the previous phases, we designed how the website looks (Frontend) and how the server processes the data (Backend), but they were operating as two separate pieces. The goal of this phase is to bridge that gap through "Form Submission and Data Insertion"

Specifically, we worked on making sure that when a user types information into our web application such as registering a new account, adding a new product to the inventory, or logging a daily sale, that data doesn't just stay on the screen. We implemented JavaScript logic using the **fetch()** API to send that information to our **Node.js** and **Express server**, which then saves it permanently into our MongoDB database. This is a huge step for **BentaBoard** because it transforms the project from a static set of pages into a working application where small sellers can actually start inputting their business data.

Objectives

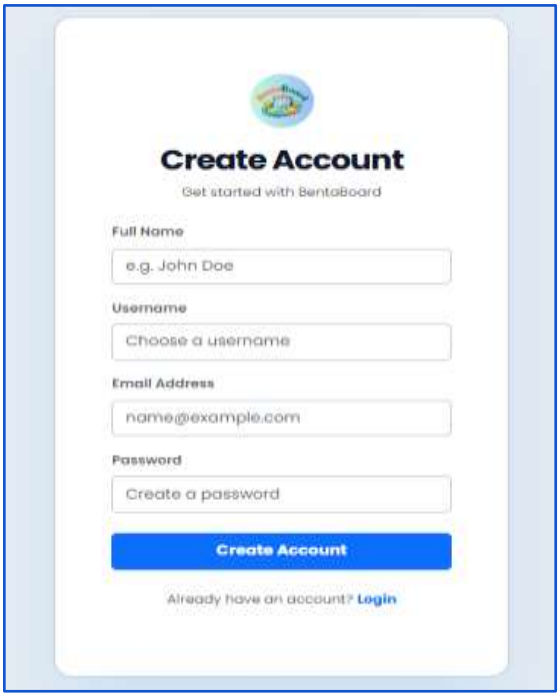
<u>At the end of this activity, we were able to:</u>

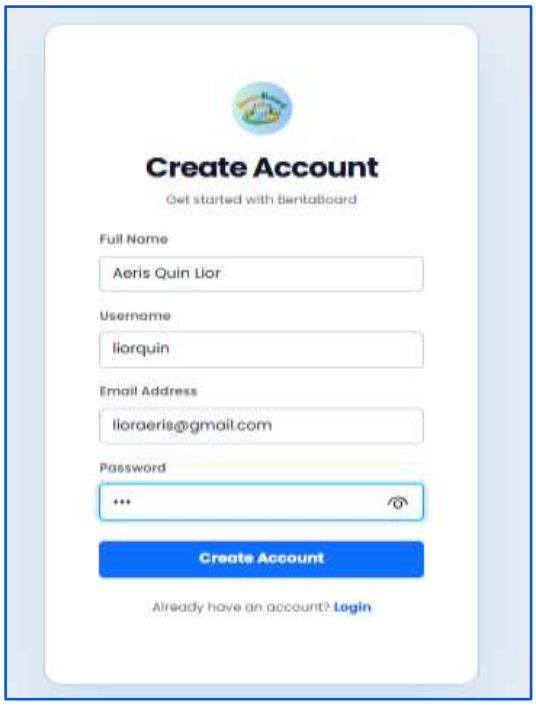
1. Link HTML forms from their frontend to the corresponding backend routes.
2. Use JavaScript fetch() or axios to handle POST requests.
3. Insert form data into MongoDB via API.
4. Validate input data before submission.
5. Display success/error messages to users based on the API response.
6. Maintain good commit hygiene and push working updates to GitHub.

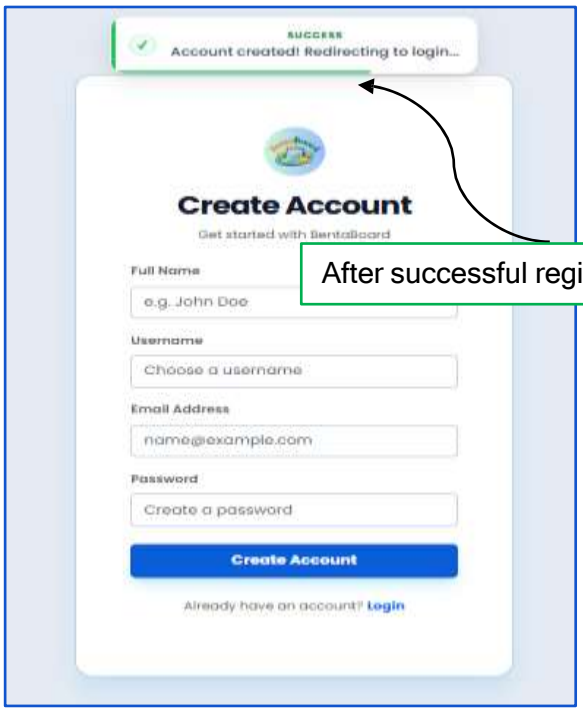
I. Screenshot of the HTML form interface designated only for Sellers:

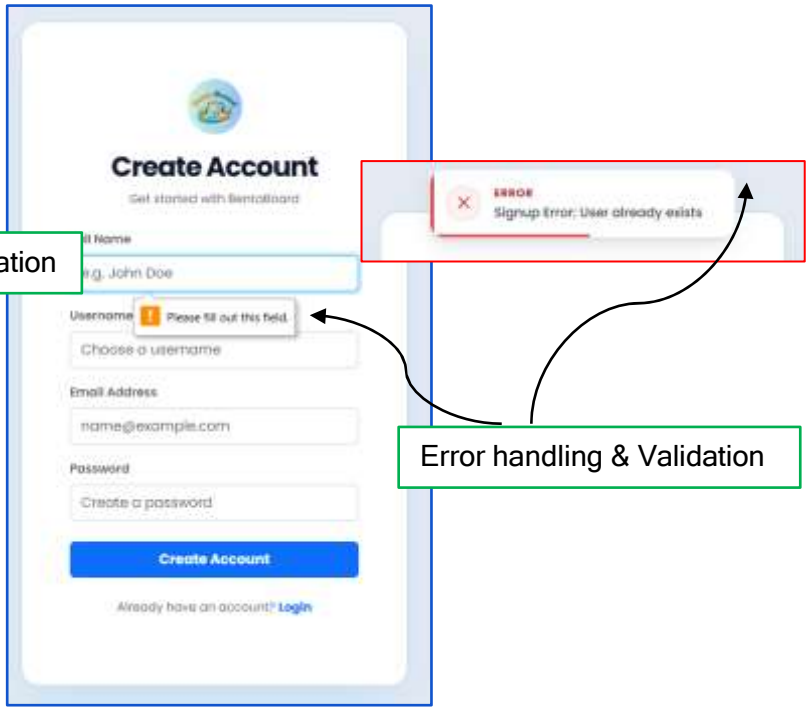
- The Screenshots attached below shows the frontend HTML forms containing error handling and validation via live server in VS Code.

A. Register Form - register.html





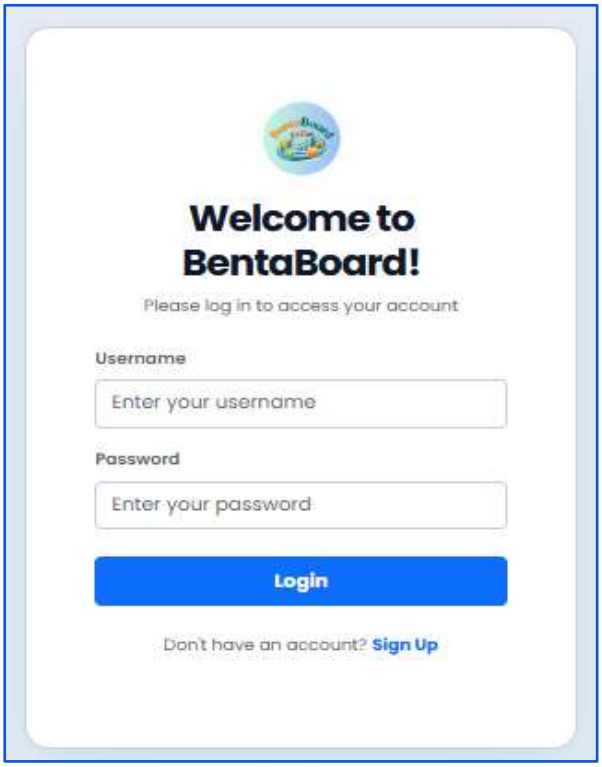


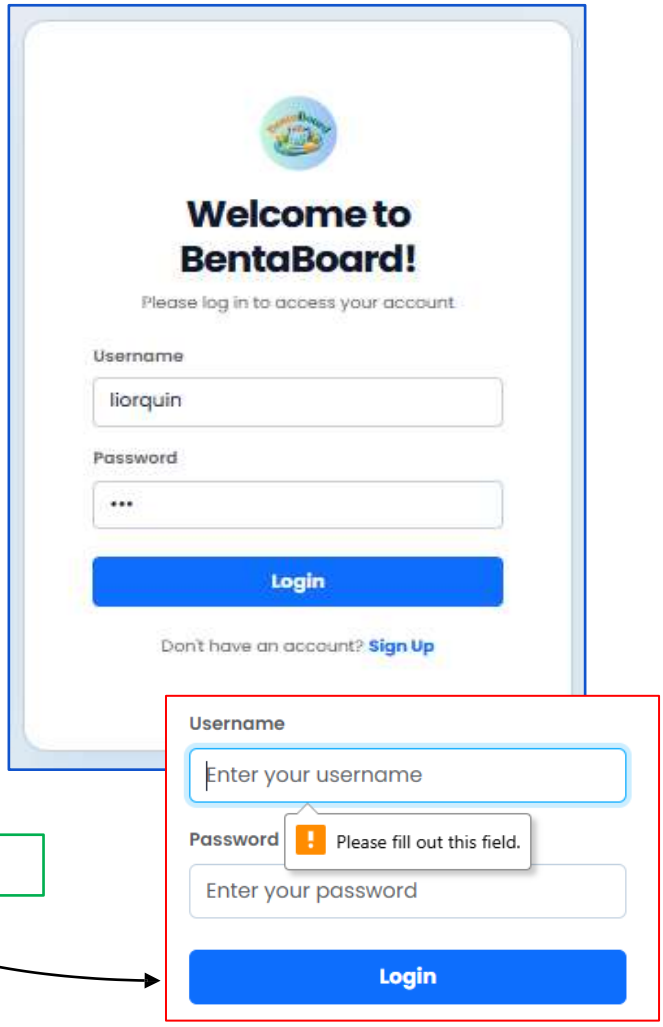


MongoDB Data Inserted

```
_id: ObjectId('69e85c5478c254e9cc18eb05')
username : "liorquin"
email : "lioraeris@gmail.com"
password : "$2b$10$CecYhjBT.3C1Kxfk6Y.EcORV/bYMmy3XqXf7u6OoJ7IkwQYh5azyG"
full_name : "Aeris Quin Lior"
role : "Seller"
created_at : 2026-04-22T05:27:48.052+00:00
```

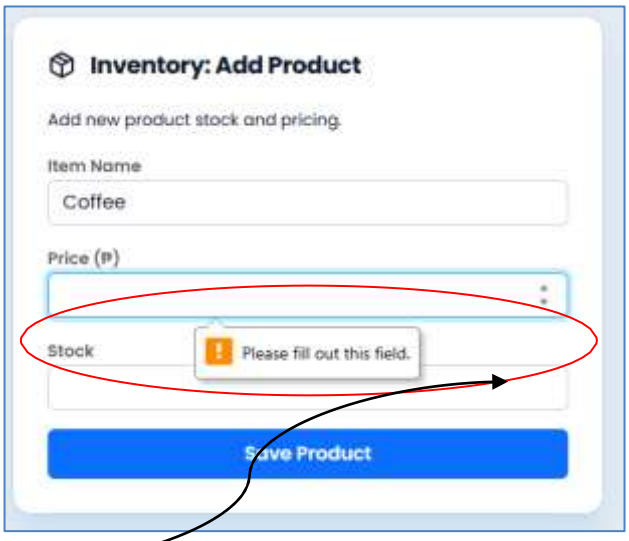
B. Log in Form - login.html





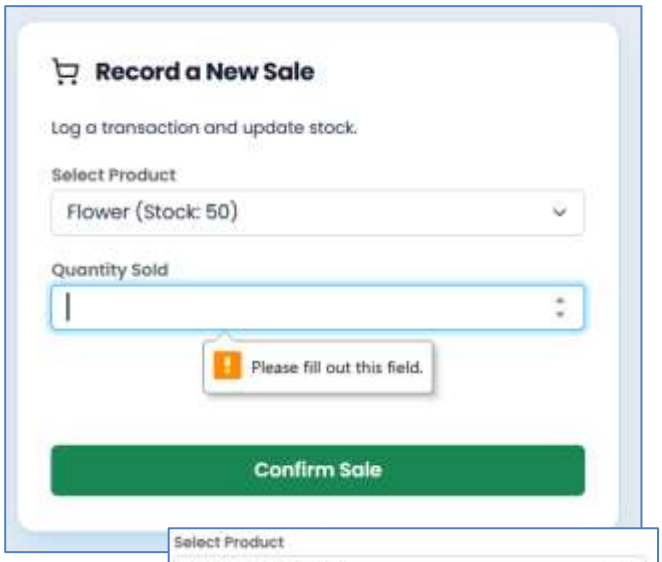
Same feedback applied on login part

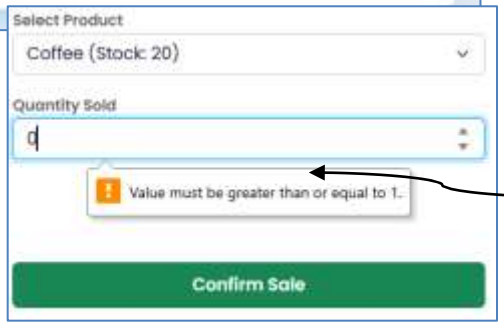
C. Adding Product, New sale, & Log expenses – dashboard.html



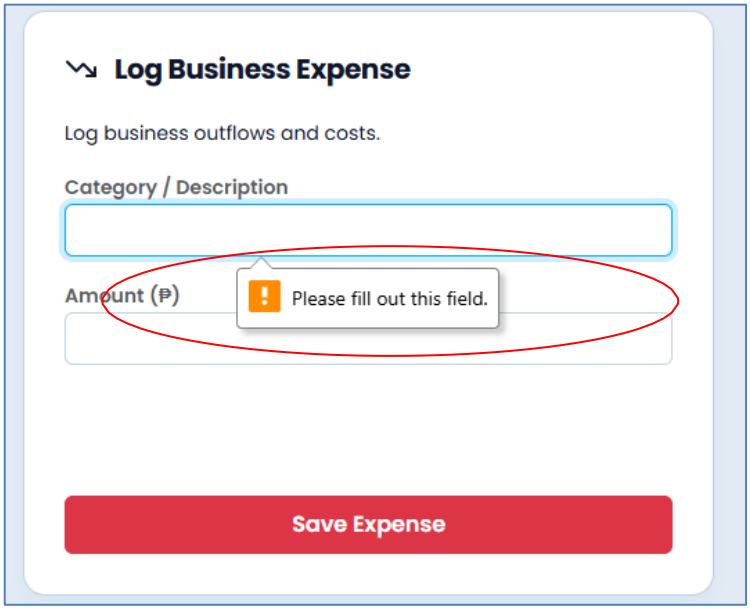
Error Handling & validation

Successful notification after added products, sale and expenses had been saved





Validate that the Price field and quantity must be at least 1 or more



MongoDB data insertion for Adding products, recording new sales and tracking log expenses:

Added **products** data inserted to MongoDB


```
_id: ObjectId('69e8ea0d0f2ce77af73941bd')
userId : ObjectId('69e5b25e95fa776e693e5acf')
name : "Coffee"
price : 49
stock : 20
createdAt : 2026-04-22T15:32:29.673+00:00
__v : 0

_id: ObjectId('69e8ea2a0f2ce77af73941be')
userId : ObjectId('69e5b25e95fa776e693e5acf')
name : "Flower"
price : 29
stock : 35
createdAt : 2026-04-22T15:32:58.689+00:00
__v : 0
```

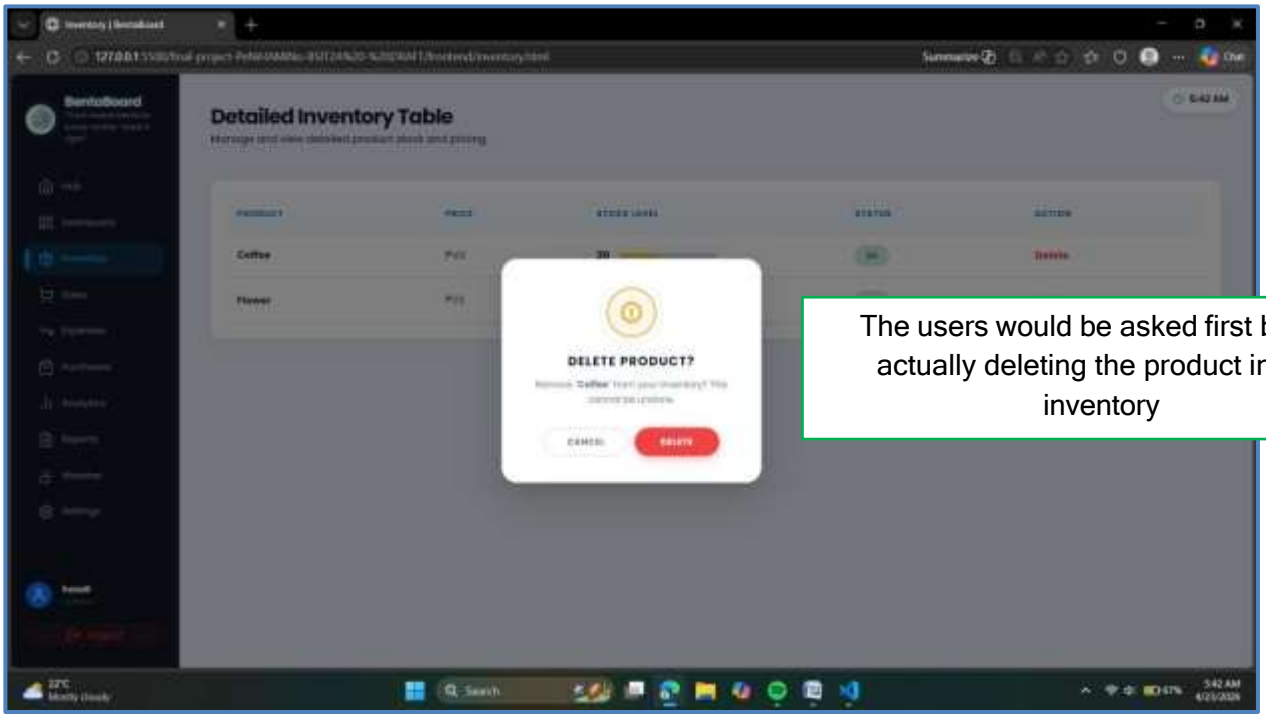

Recorded **sales** data inserted to MongoDB


```
_id: ObjectId('69e84e408ee371239d6a35ff')
userId : ObjectId('69e5b25e95fa776e693e5acf')
productName : "Yakult"
quantity : 80
total : 960
saleDate : 2026-04-22T04:27:44.209+00:00
__v : 0

_id: ObjectId('69e8ea460f2ce77af73941bf')
userId : ObjectId('69e5b25e95fa776e693e5acf')
productName : "Flower"
quantity : 15
total : 435
saleDate : 2026-04-22T15:33:26.937+00:00
__v : 0
```

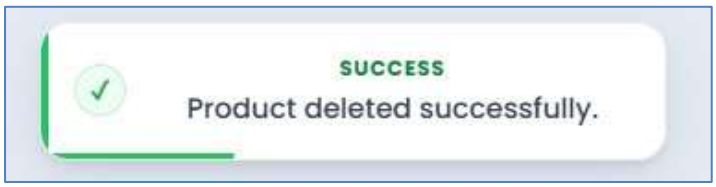

Recorded **purchases** data inserted to MongoDB


```
_id: ObjectId('69e84e5a0ee371239d6a3600')
userId : ObjectId('69e5b25e95fa776e693e5acf')
category : "Transportation"
amount : 20
dateLogged : 2026-04-22T04:28:10.511+00:00
__v : 0

_id: ObjectId('69e8ea5a0f2ce77af73941c0')
userId : ObjectId('69e5b25e95fa776e693e5acf')
category : "Supply"
amount : 700
dateLogged : 2026-04-22T15:33:46.478+00:00
__v : 0
```

D. Delete Product – inventory.html



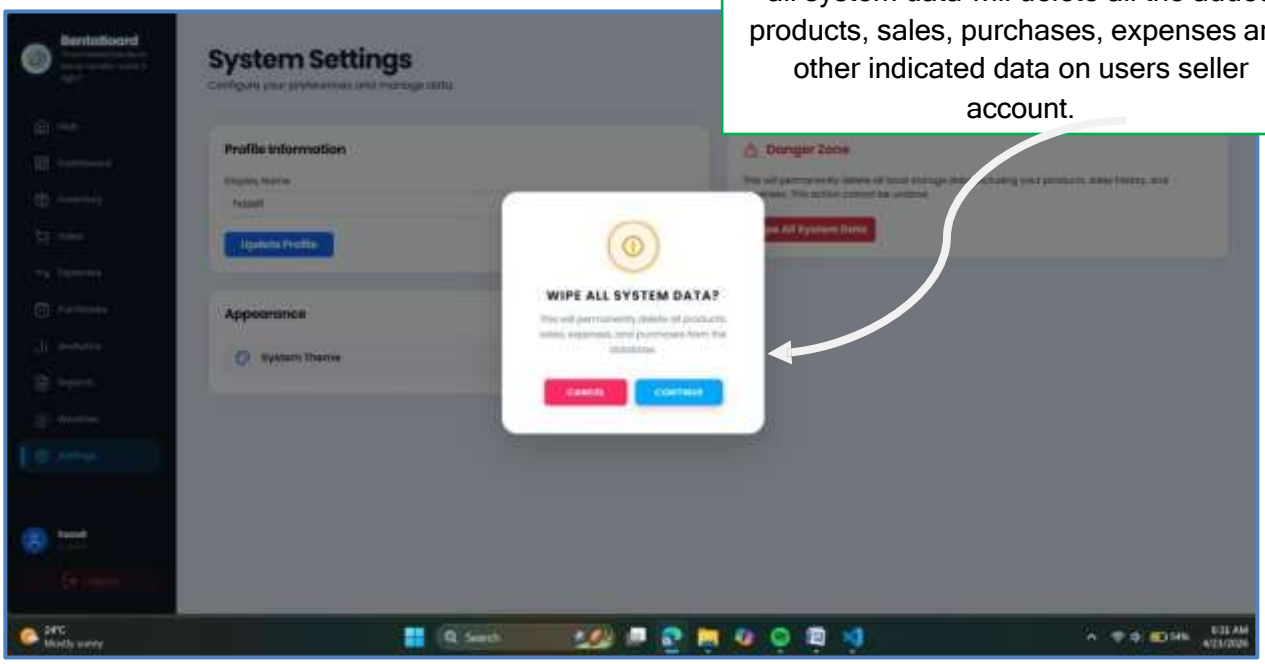
The users would be asked first before actually deleting the product inside inventory

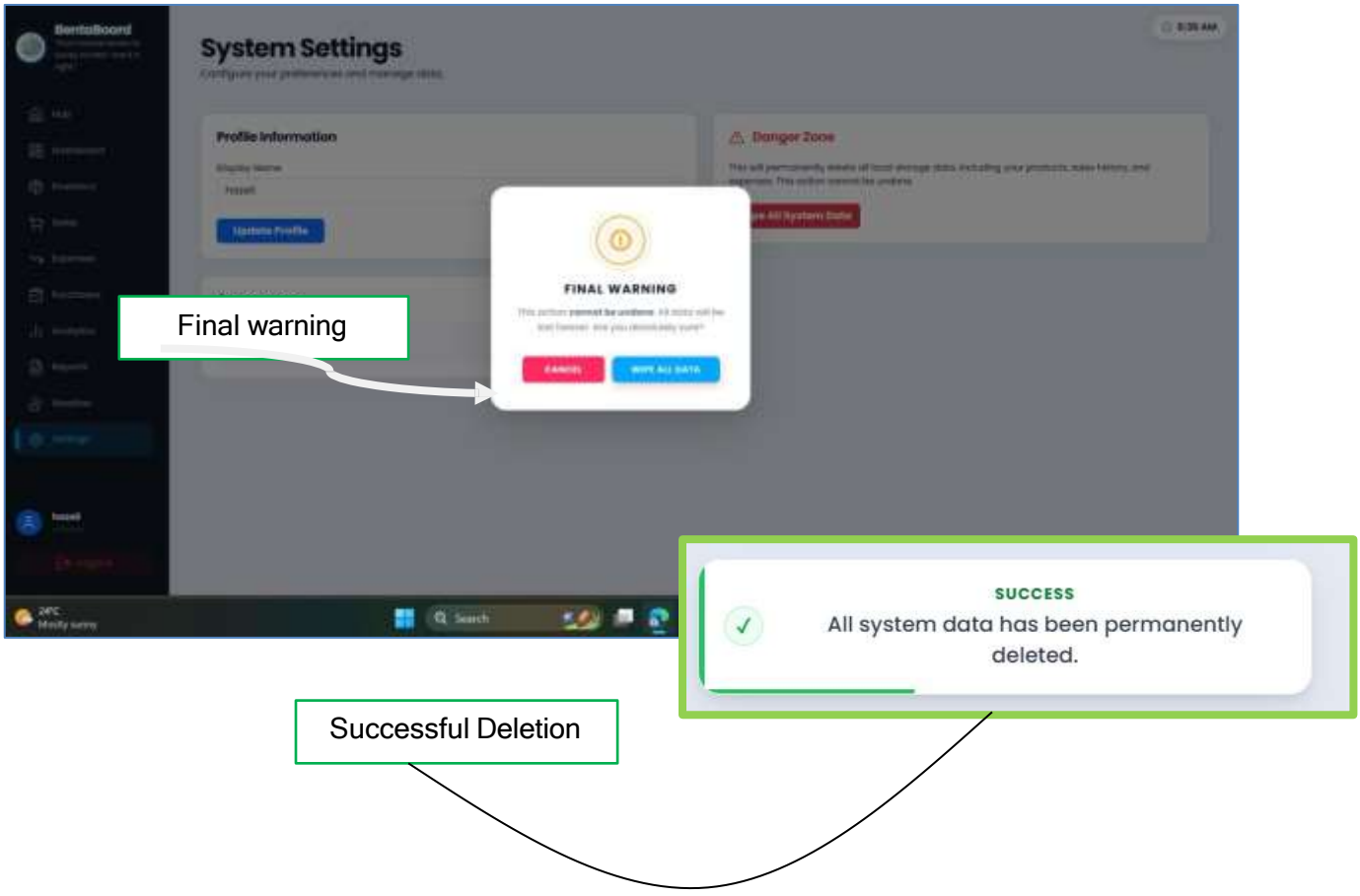


The system feedback after successfully deleting the products

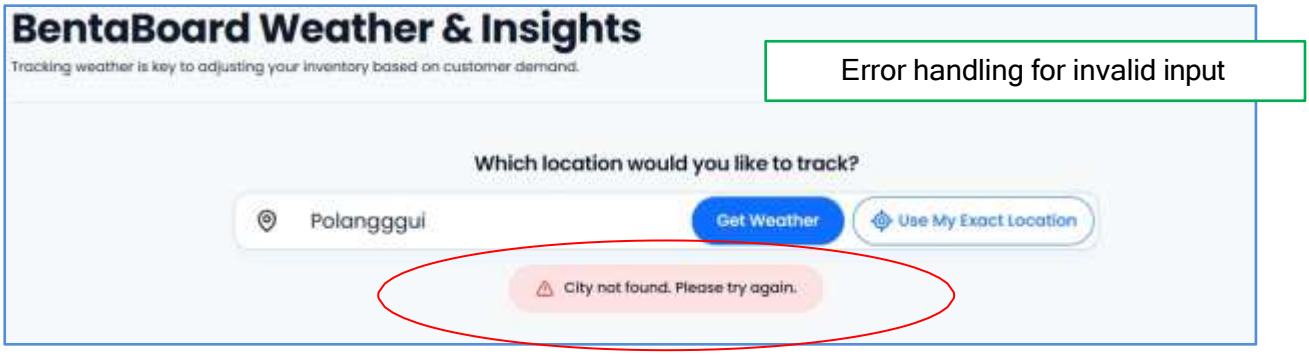
E. Wipe all Data – settings.html

The system will warn the users that wiping all system data will delete all the added products, sales, purchases, expenses and other indicated data on users seller account.

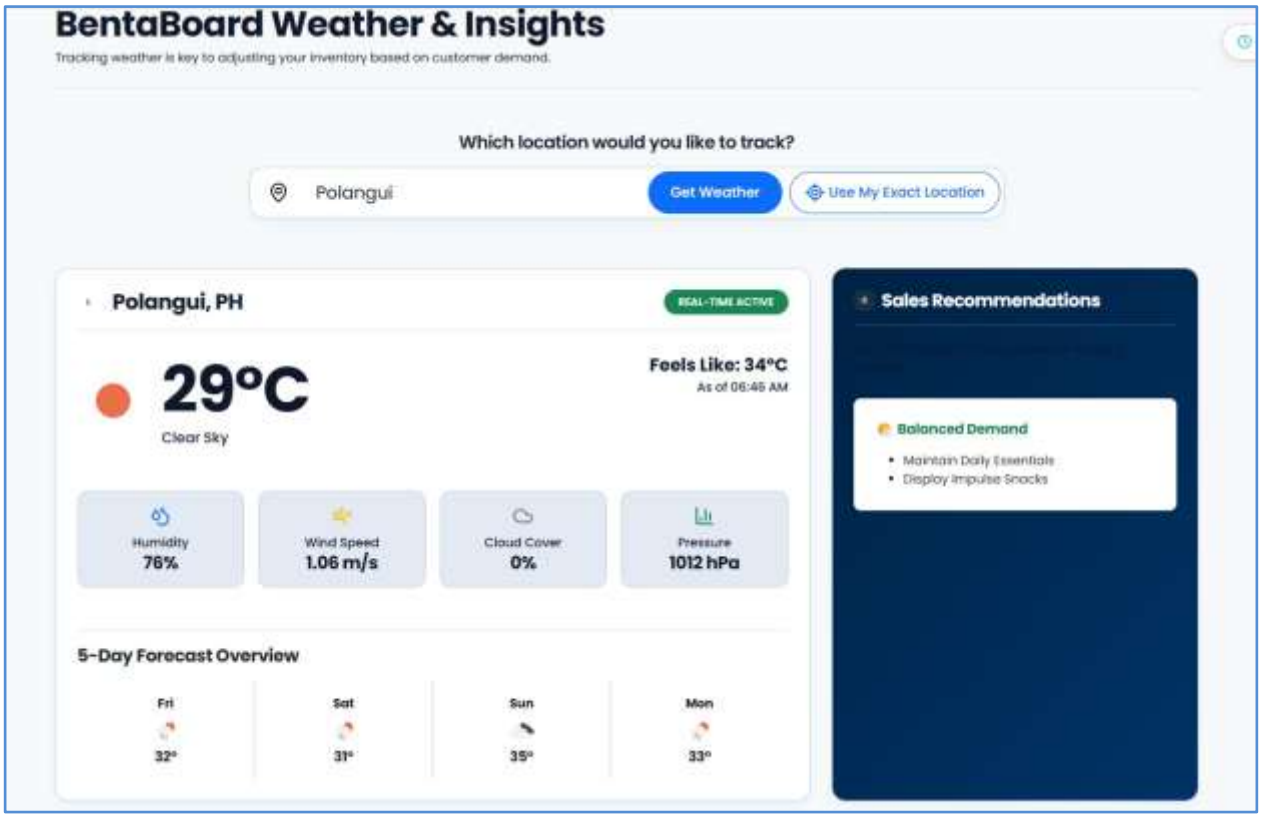




D. OpenWeatherMap fetch – weather.html



Error handling for invalid input



This will be the output if users had successfully input a correct searchable location

II. Screenshot of JS code used for API connection

1. User Registration Handler

```
async function handleAuth(e, type) {
    e.preventDefault();
    const form = e.target;
    const formData = new FormData(form);

    if (type === 'signup') {
        const userData = {
            full_name: formData.get('regName'),
            username:  formData.get('regUser'),
            email:     formData.get('regEmail'),
            password:  formData.get('regPass'),
            role:      "Seller"
        };
        try {
            const response = await fetch(`${AUTH_API_URL}/register`, {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify(userData)
            });
            const data = await response.json();
            if (response.ok) {
                bentaToast('success', 'Account created! Redirecting to login...');
                setTimeout(() => { window.location.href = 'login.html'; }, 1800);
                form.reset();
            } else {
                bentaToast('error', 'Signup Error: ' + (data.msg || data.error || 'User already exists'));
            }
        } catch (err) {
            bentaToast('error', 'Connection failed. Is your backend server running on port 3000?');
        }
```

2. User Login Handler

```
    async function handleAuth(e, type) {
    const loginData = {
        username: formData.get('loginUser'),
        password: formData.get('loginPass')
    };
    try {
        const response = await fetch(`${AUTH_API_URL}/login`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(loginData)
        });
        const data = await response.json();
        if (response.ok) {
            localStorage.setItem('token', data.token);
            localStorage.setItem('bb_user', data.user.username);
            localStorage.setItem('bb_welcome_triggered', 'true');
            window.location.href = 'index.html';
        } else {
            bentaToast('error', 'Login Failed: ' + (data.msg || 'Invalid credentials'));
        }
    } catch (err) {
        bentaToast('error', 'Could not connect to the server.');
    }
}
}
```

3. Add Product Handler

```
async function addProduct(e) {
    e.preventDefault();
    const form      = e.target;
    const nameInput  = form.querySelector('[name="pName"]');
    const priceInput = form.querySelector('[name="pPrice"]');
    const stockInput = form.querySelector('[name="pStock"]');

    if (nameInput?.value.trim() && priceInput?.value > 0 && stockInput?.value > 0) {
        const stockQty    = parseInt(stockInput.value);
        const productData = {
            name:  nameInput.value.trim(),
            price: parseFloat(priceInput.value),
            stock: stockQty
        };
        try {
            const response = await fetch(`${API_BASE_URL}/products`, {
                method:  'POST',
                headers: getAuthHeaders(),
                body:    JSON.stringify(productData)
            });
            if (response.ok) {
                const saved = await response.json();
                products.push({ _id: saved.product._id, ...productData, initialStock: stockQty });
                saveData();
                renderInventory();
                populateSaleSelect();
                form.reset();
                bentaToast('success', 'Product has been saved!');
            }
        } catch (error) {
            bentaToast('error', 'Error: Check if server is running on port 3000.');
        }
    }
}
```

4. Record Sale Handler

```
async function recordSale(e) {
    e.preventDefault();
    const form       = e.target;
    const saleSelect = document.querySelector('#saleSelect, #dashboardSaleSelect');
    const qtyInput   = form.querySelector('[name="saleQty"]');
    const index      = saleSelect.value;
    const qty        = parseInt(qtyInput ? qtyInput.value : 0);

    if (index !== "" && products[index] && qty > 0 && products[index].stock >= qty) {
        const saleData = { productName: products[index].name, quantity: qty };
        try {
            const response = await fetch(`${API_BASE_URL}/sales`, {
                method:  'POST',
                headers: getAuthHeaders(),
                body:    JSON.stringify(saleData)
            });
            if (response.ok) {
                const total = products[index].price * qty;
                products[index].stock -= qty;
                salesLog.push({
                    date:  new Date().toLocaleDateString(),
                    name:  products[index].name,
                    qty:   qty,
                    total: total
                });
                saveData();
                renderSalesLog();
                populateSaleSelect();
                form.reset();
                bentaToast('success', 'Sale has been saved!');
            } else {
                bentaToast('error', 'Failed to save sale.');
            }
        } catch (error) {
            bentaToast('error', 'Fatal Error: Could not connect to backend.');
        }
    } else {
        bentaToast('warning', 'Insufficient stock or invalid product selection!');
    }
}
```

5. Log Expense Handler

```
async function logExpense(e) {
    e.preventDefault();
    const form          = e.target;
    const categoryInput = form.querySelector('[name="expCat"]');
    const amountInput   = form.querySelector('[name="expAmt"]');

    if (categoryInput?.value.trim() && amountInput?.value > 0) {
        const expenseData = {
            category: categoryInput.value.trim(),
            amount:   parseFloat(amountInput.value)
        };
        try {
            const response = await fetch(`${API_BASE_URL}/expenses`, {
                method:  'POST',
                headers: getAuthHeaders(),
                body:    JSON.stringify(expenseData)
            });
            if (response.ok) {
                expenseLog.push({ date: new Date().toLocaleDateString(), ...expenseData });
                saveData();
                renderExpenseLog();
                form.reset();
                bentaToast('success', 'Expense has been saved!');
            }
        } catch (error) {
            bentaToast('error', 'Connection Error: Server might be down.');
        }
    }
}
```

6. Delete Product Handler

```
async function deleteProduct(index) {
    bentaNotify.confirm(
        'Delete Product?',
        `Remove "<strong>${products[index].name}</strong>" from your inventory? This cannot be undone.`,
        'Delete',
        async () => {
            const productId = products[index]._id;
            if (productId) {
                try {
                    await fetch(`${API_BASE_URL}/products/${productId}`, {
                        method:  'DELETE',
                        headers: getAuthHeaders()
                    });
                } catch (err) {
                    console.warn('Could not delete from DB:', err);
                }
            }
            products.splice(index, 1);
            saveData();
            renderInventory();
            populateSaleSelect();
            bentaToast('success', 'Product deleted successfully.');
        }
    );
}
```

7. Load Purchases Handler

```
async function renderPurchasesTable() {
    if (!purchasesLogBody) return;
    try {
        const response  = await fetch(`${API_BASE_URL}/purchases`, { headers: getAuthHeaders() });
        const purchases = await response.json();
        if (purchases.length === 0) {
            purchasesLogBody.innerHTML = `<tr><td colspan="6" class="text-center text-muted py-4">No recorded purchase yet.</td></tr>`;
            return;
        }
        purchasesLogBody.innerHTML = purchases.map(p => `
            <tr>
                <td>${new Date(p.purchaseDate).toLocaleDateString()}</td>
                <td class="fw-medium">${p.supplierName}</td>
                <td>${p.productName}</td>
                <td>${p.quantity}</td>
                <td>₱${parseFloat(p.unitCost).toFixed(2)}</td>
                <td class="fw-bold text-dark">₱${parseFloat(p.totalCost).toFixed(2)}</td>
            </tr>`).join('');
    } catch (error) {
        purchasesLogBody.innerHTML = `<tr><td colspan="6" class="text-center text-danger py-4">Error loading data from server.</td></tr>`;
    }
}
```

8. Add Purchase Handler

```
purchaseForm.addEventListener('submit', async (e) => {
    e.preventDefault();
    const purchaseData = {
        productName:  document.getElementById('productId').value,
        supplierName: document.getElementById('supplierName').value,
        quantity:     parseInt(document.getElementById('quantity').value),
        unitCost:     parseFloat(document.getElementById('unitCost').value),
        totalCost:    parseFloat(totalCostDisplay.textContent),
        purchaseDate: document.getElementById('purchaseDate').value || new Date()
    };
    try {
        const response = await fetch(`${API_BASE_URL}/purchases/add`, {
            method:  'POST',
            headers: getAuthHeaders(),
            body:    JSON.stringify(purchaseData)
        });
        if (response.ok) {
            bentaToast('success', 'Purchase has been saved!');
            purchaseForm.reset();
            document.getElementById('purchaseDate').valueAsDate = new Date();
            totalCostDisplay.textContent = "0.00";
            renderPurchasesTable();
        } else {
            const err = await response.json();
            bentaToast('error', `Error: ${err.message}`);
        }
    } catch (error) {
        bentaToast('error', 'Backend connection failed.');
    }
});
}
```

9. Wipe All Data Handlers

```
async function wipeAllSystemData() {
    bentaNotify.confirm(
        'Wipe All System Data?',
        'This will permanently delete all products, sales, expenses, and purchases from the database.',
        'Continue',
        () => {
            bentaNotify.confirm(
                'Final Warning',
                'This action <strong>cannot be undone</strong>. All data will be lost forever. Are you absolutely sure?',
                'Wipe All Data',
                async () => {
                    try {
                        const response = await fetch(`${API_BASE_URL}/data/wipe`, {
                            method: 'DELETE',
                            headers: getAuthHeaders()
                        });

                        if (response.ok) {
                            localStorage.removeItem(STORAGE_KEYS.products);
                            localStorage.removeItem(STORAGE_KEYS.sales);
                            localStorage.removeItem(STORAGE_KEYS.expenses);

                            products   = [];
                            salesLog   = [];
                            expenseLog = [];

                            if (document.getElementById('fullInventoryBody')) renderInventory();
                            if (document.getElementById('salesLogBody'))      renderSalesLog();
                            if (document.getElementById('expenseLogBody'))    renderExpenseLog();
                            if (document.getElementById('displaySales'))      updateDashboard(currentFilter);
                            if (document.getElementById('reportSalesTotal'))  generateReport(currentFilter);
                            if (document.querySelector('select[name="saleSelect"], #saleSelect, #dashboardSaleSelect')) {
                                populateSaleSelect();
                            }
                            if (salesChartInstance) renderChart(currentFilter);
```

```
                            bentaToast('success', 'All system data has been permanently deleted.');
                        } else {
                            const err = await response.json();
                            bentaToast('error', 'Failed to wipe data: ${err.message}');
                        }
                    } catch (error) {
                        console.error('Wipe error:', error);
                        bentaToast('error', 'Could not connect to server. Please check your connection.');
                    }
                }
            );
        }
    );
}
```

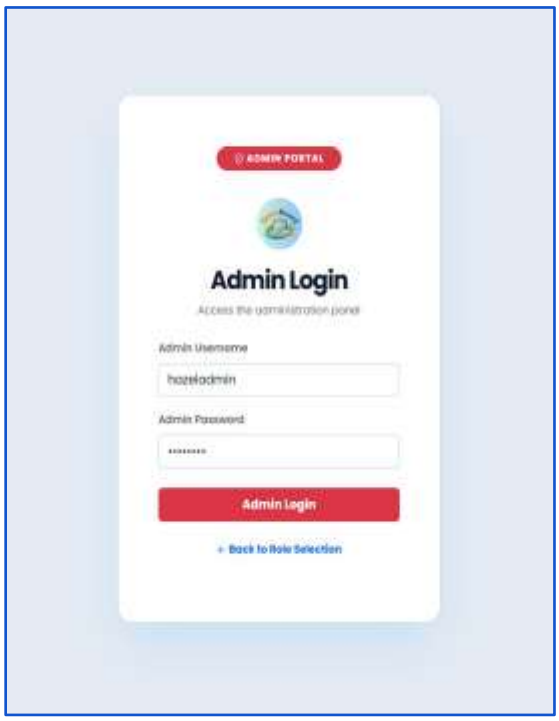
10. OpenWeatherMap Fetch Handler

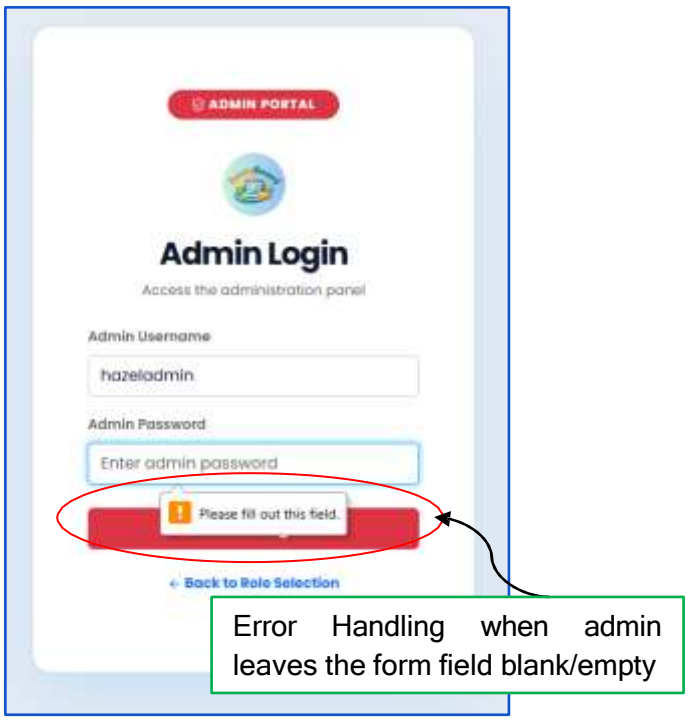
```
async function fetchWeatherData(lat, lon, label) {
    toggleLoading(true);
    try {
        const [wRes, fRes] = await Promise.all([
            fetch(`https://api.openweathermap.org/data/2.5/weather?lat=${lat}&lon=${lon}&units=metric&appid=${API_KEY}`),
            fetch(`https://api.openweathermap.org/data/2.5/forecast?lat=${lat}&lon=${lon}&units=metric&appid=${API_KEY}`)
        ]);

        if (!wRes.ok) throw new Error("City not found. Please try again.");

        updateUI(await wRes.json(), await fRes.json(), label);
        displayArea.style.display = "flex";
        displayArea.classList.add('weather-reveal');
    } catch (err) {
        triggerError(err.message);
    } finally {
        toggleLoading(false);
    }
}
```

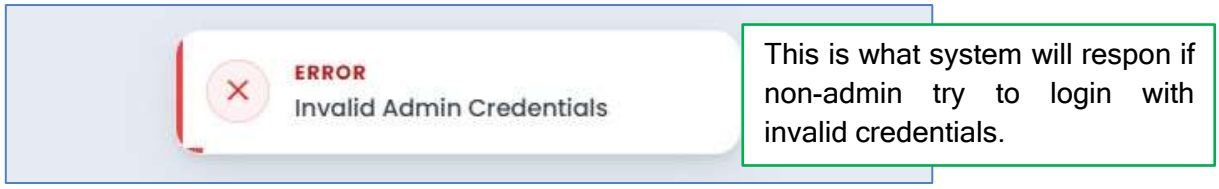
11. Weather Geocoding Handler

```
async function fetchByCity(cityName) {
    toggleLoading(true);
    try {
        const geoRes = await fetch(`https://api.openweathermap.org/geo/1.0/direct?q=${cityName}&limit=1&appid=${API_KEY}`);
        const geoData = await geoRes.json();
        if (!geoData.length) throw new Error("City not found. Please try again.");

        const { lat, lon, name, country } = geoData[0];
        await fetchWeatherData(lat, lon, `${name}, ${country}`);
    } catch (err) {
        triggerError(err.message);
    }
}
```

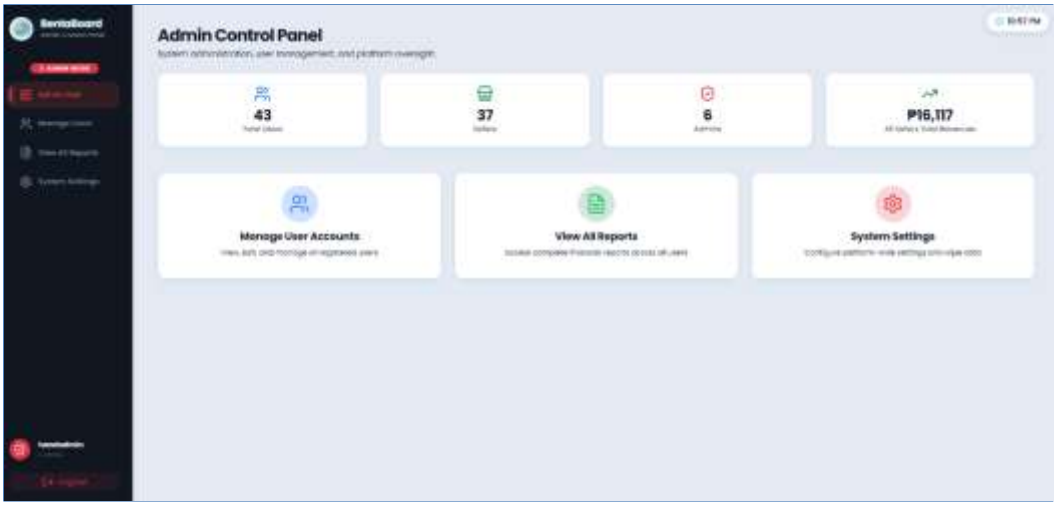
Screenshot of the HTML form interface designated only for Admin.







- **Admin view after successfully logged in (Full overview seat on Admin Hub)**



IV. Screenshot of GitHub commits (frontend and backend)

Note:

In this commit log, we linked HTML forms from their frontend to the corresponding backend routes and implemented error handling.

V. Individual reflections from each group member (3–5 sentences)

Hazel Ann Obidece - Project Manager & Backend Developer

As the project manager, I had to make sure the backend was fully ready and stable before the frontend developer started linking the forms. I spent most of my time ensuring the POST routes were working and that the server could handle incoming data without crashing. This phase taught me how to coordinate different parts of the project to make sure everything fits together perfectly.

Noe Jr Remolar - Documentation Officer

My main focus was writing the actual logic for the API endpoints to ensure that the data was being received and processed correctly. I learned that even a small typo in a route or a missing header can break the entire connection between the user and the server. It was a great experience learning how to troubleshoot these errors to make the system more reliable.

Michael Mantes - Database Manager

My task was to monitor MongoDB Atlas and verify that the data coming from the forms actually matched our predefined schemas. It was very satisfying to see the collections fill up with real data and know that our database structure was working as intended. This process helped me understand how to handle data validation to prevent errors from entering the database.

Nica Alyssa Racadag - FrontEnd Manager

My task is to handle the 'bridge' between the user and the server by writing the JavaScript `fetch()` calls and managing the form submissions. The most challenging part was ensuring that the form data was captured accurately and that the user received a clear alert message for every action. I learned how important it is to provide immediate feedback to the user so they aren't left guessing if the page worked.

Pear Anne Odo - GitHub Manager

Since Phase 4 involved frequent changes to both the frontend and backend folders, my job was to ensure that no one's code was accidentally overwritten during the merge. I focused on keeping our commit history organized and writing clear messages so the team could track our progress. This phase taught me a lot about version control and the importance of a clean workflow in a group project.

NOTE:

AI Usage: *On creating this Phase 3 Lab Report (documentation), around 20% of AI usage was obtained to guide us on finishing this report such as: asking assistance when encountering errors and other suggestions and recommendations that would serve as our guide for improvement. Overall, we hereby acknowledge that all the information and key ideas presented on this phase was based on our actual work, outputs, and understanding of the concepts.*